



University
of Basel

Chapter 1: Introduction

Foundations of Distributed Systems

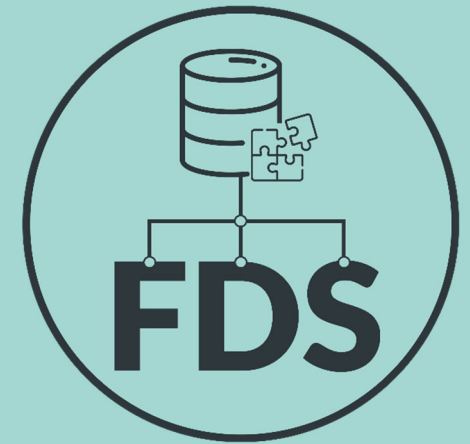
Fall Semester 2025

Prof. Dr. Florina Ciorba

Prof. Dr. Heiko Schuldt

Prof. Dr. Christian Tschudin

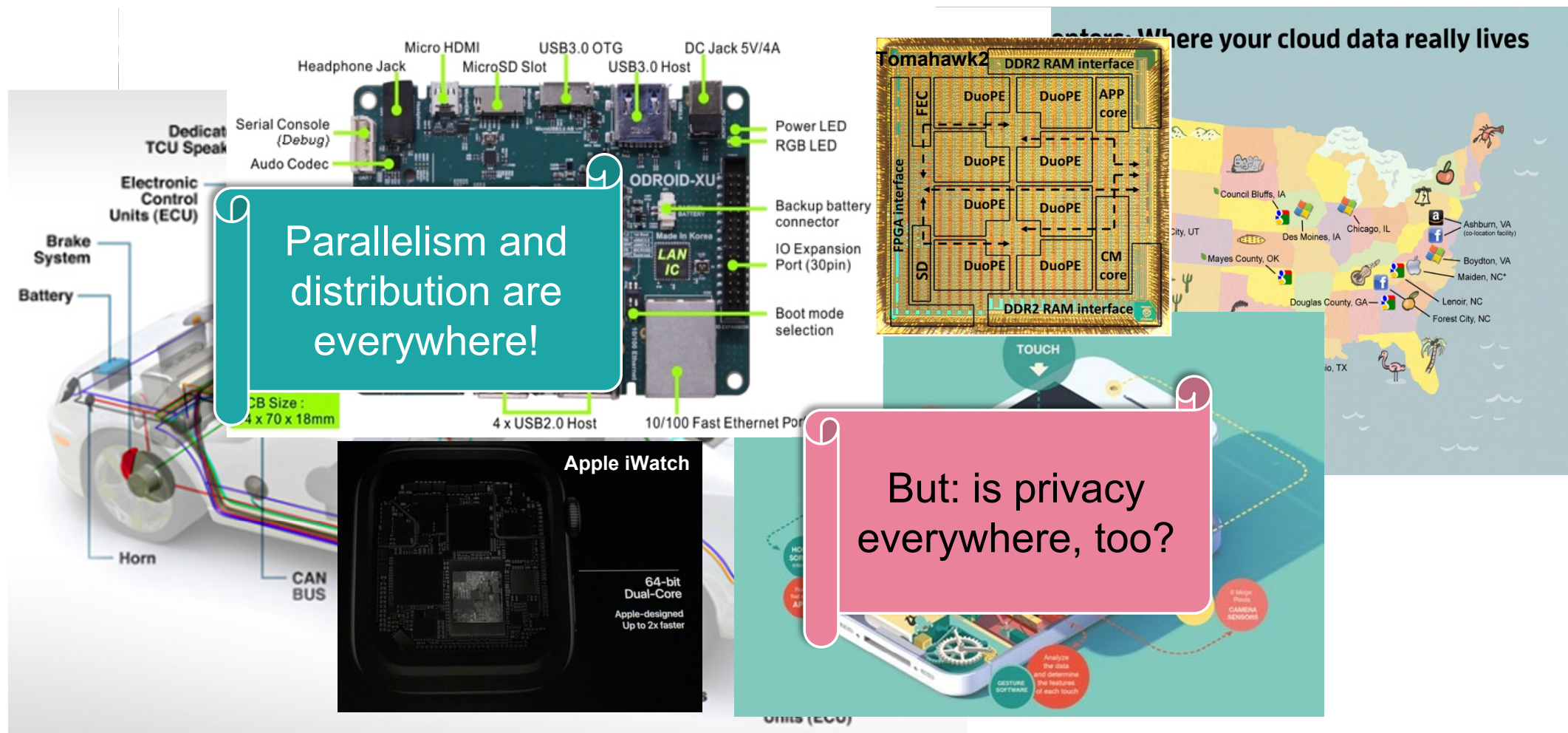
Prof. Dr. Isabel Wagner



Overview of Chapter 1

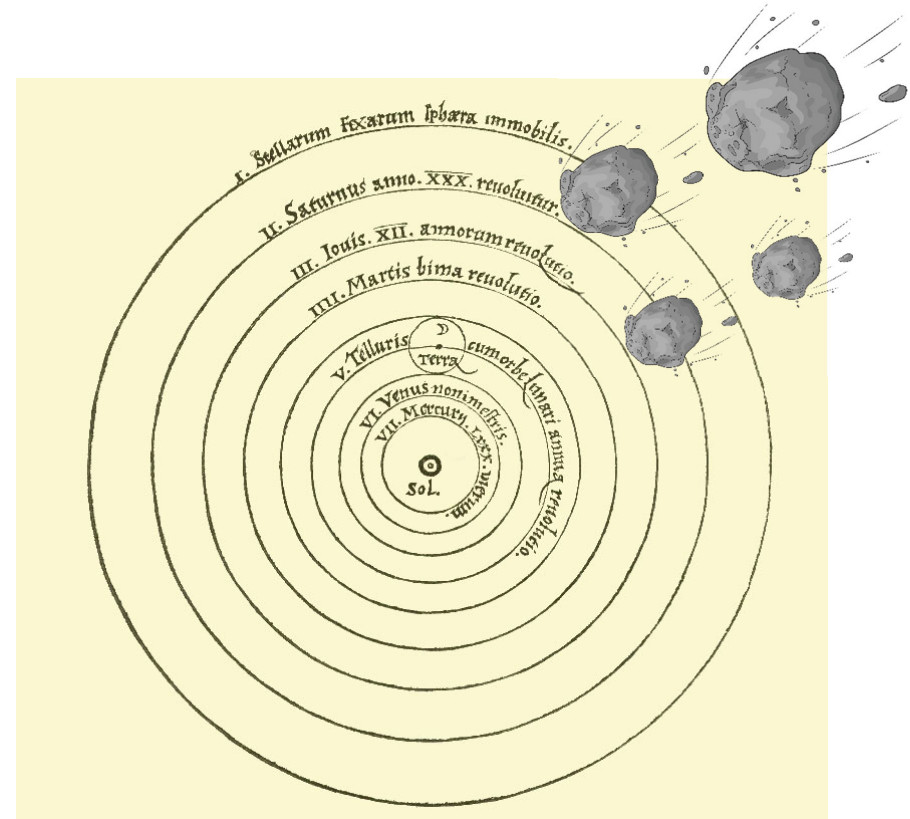
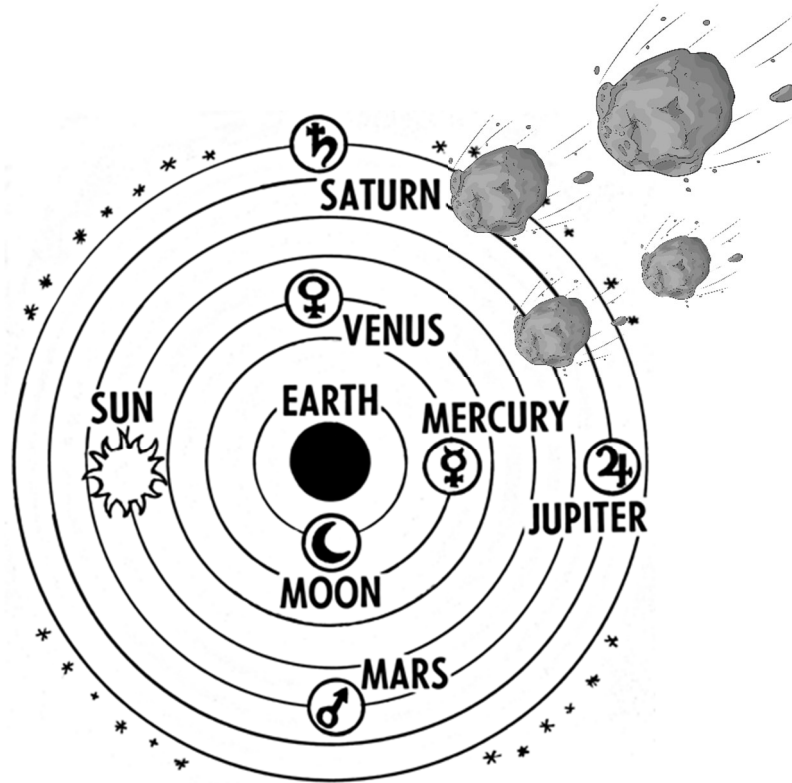
- | | |
|-----|---|
| 1.1 | What are parallel and distributed systems? |
| 1.2 | ... from a Communication Perspective |
| 1.3 | ... from a Data Perspective |
| 1.4 | ... from a Processing Perspective |
| 1.5 | ... from a Privacy Perspective |
| 1.6 | Again: what are Parallel and Distributed Systems? |

1.1 What are Parallel and Distributed Systems?



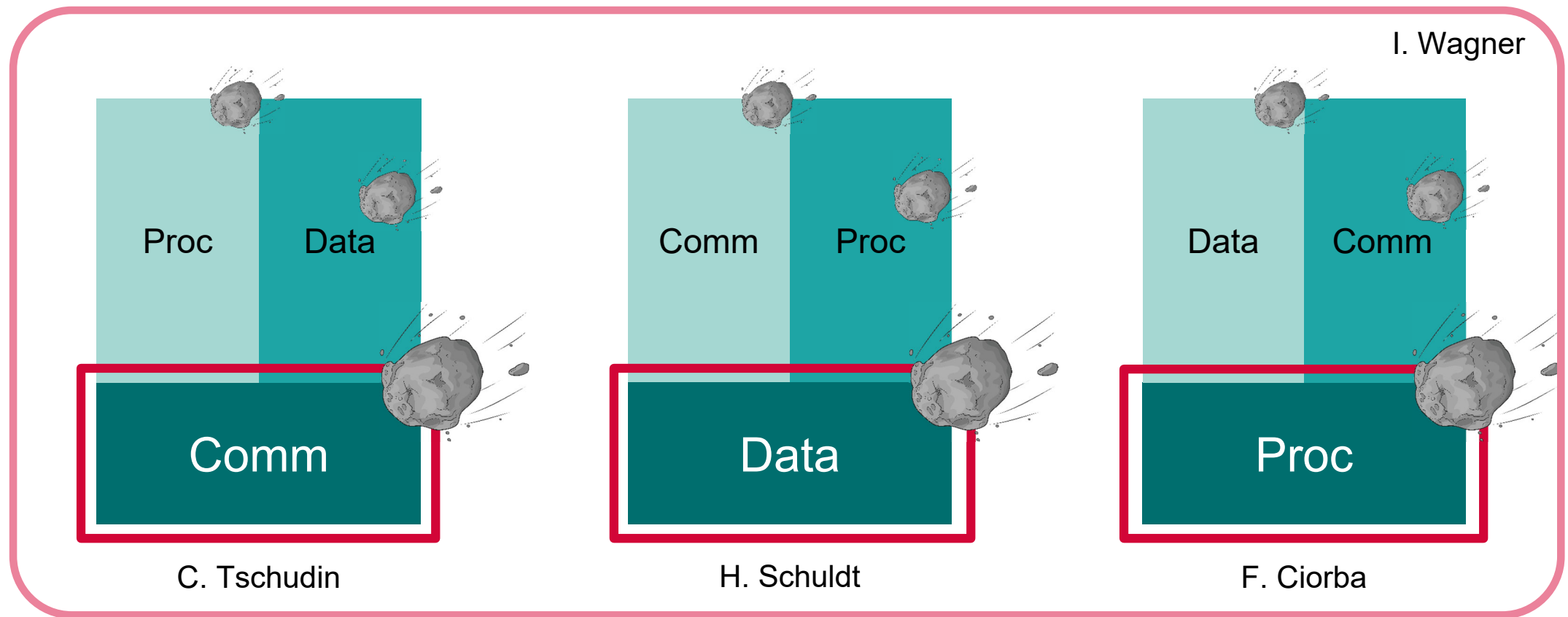
Ptolemaic vs Copernican View of the World

i.e., Communication/Processing/Data (comm/proc/data)



- Privacy can be considered as a swarm of Asteroids challenging all planets

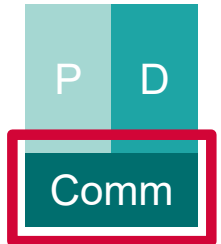
Where the Sun is (or the Earth) and where the Threat comes in



Overview of Chapter 1

-
- | | |
|-----|---|
| 1.1 | What are parallel and distributed systems? |
| 1.2 | ... from a Communication Perspective |
| 1.3 | ... from a Data Perspective |
| 1.4 | ... from a Processing Perspective |
| 1.5 | ... from a Privacy Perspective |
| 1.6 | Again: what are Parallel and Distributed Systems? |
-

1.2 **What are** parallel and distributed systems, from a Communications Perspective?



Distributed computer systems

- did not exist 50 years ago *because computer networks did not exist, back then*
- did not work well, 40 years ago *lack of protocol design theory*
- did require hefty servers, 30 years ago *peer-to-peer was not invented yet*
- did not scale well, 20 years ago *the cloud did not exist yet*
- found themselves in a centrality black hole, 10 years ago *algorithmic and economic deficits*

1.2 **What are** parallel and distributed systems, from a Communications Perspective?

Distributed computer systems

- did not exist 50 years ago

- did not work well

- did require hefty

- did not scale well,

- found themselves in

tremendous progress over 50 years
 (your powerful smartphone
 is just the tip of an iceberg,
 sits on the top of an humongous
 technology pyramid)

not exist, back then

ol design theory

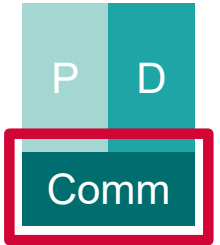
not invented yet

the cloud did not exist yet

hole, 10 years ago

algorithmic and economic deficits

1.2 **What are** parallel and distributed systems, from a Communications Perspective?



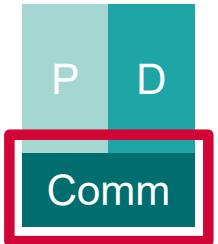
tremendous progress over 50 years
(your powerful smartphone
is just the tip of an iceberg,
sits on the top of an humongous
technology pyramid)

This means for your education:

- ways too much innovation and knowhow to absorb
- scientific shakeout is still ongoing
("What will survive the test of time?")
- **foundations** will not go away, we have to work out the big lines

"So what you say is: **it all started with computer networks**, right?"

1.2 What are parallel and distributed systems, from a Communications Perspective?



Distributed “Computers” DID exist, 80 years ago



Distributed “computers” at NASA Jet Propulsion Laboratory, starting in 1936. Calculated launch windows, trajectories, fuel consumption of rockets...

Other human computers received computations problems and sent back results by mail!

Some were making mistakes and others might be interrupted and not complete their task!

➡ **Central** steering and funding: “command-control” context.
□ Computer Science is about automating ... also this

Going back to the Basics, Prepare for the Future

P D

Comm

Progress does not stop here or today: stay tuned, prepare to learn even more!

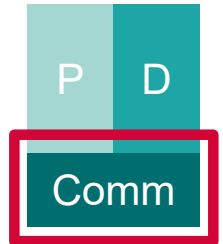
In FDS.Comm you will learn about **distribution concepts** useful for both centralized (cloud) and decentralized settings:

- **Concurrency** and why it is hard, theoretical limits of distributed systems, failure modes
- How to work *without* precise time, how to **scale**, how to persist data in a **crash-resilient** way, how to handle **malicious actors**, how to **coordinate** work in a distributed system without bottleneck **consensus**



main take-home message

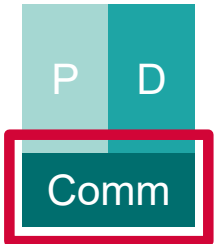
focus is on “Distribution”
although area label is “Comm”



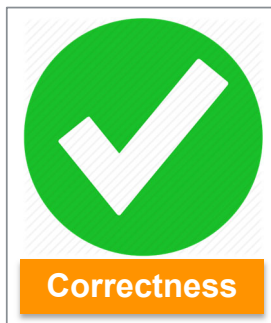
Distributed computer systems made tremendous progress, as it

- did not exist **50 years ago** *because computer networks did not exist, back then*
- did not work well, **40 years ago** *lack of protocol design theory*
- did require hefty servers, **30 years ago** *peer-to-peer was not invented yet*
- did not scale well, **20 years ago** *the cloud did not exist yet*
- found themselves in a centrality black hole, **10 years ago** *algorithmic and economic deficits*

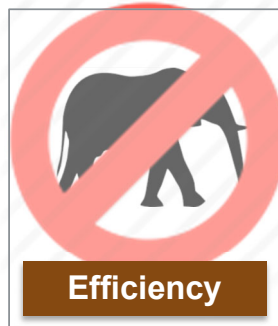
From a Communication Perspective: Goals & Challenges of Distributed Systems



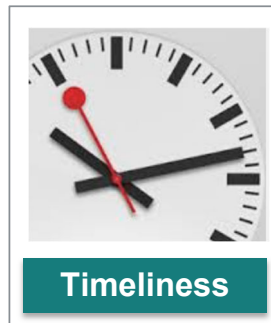
Goals



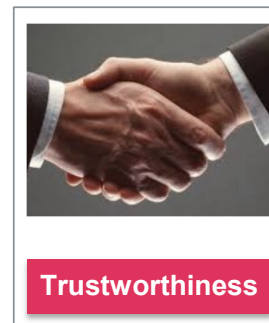
Correctness



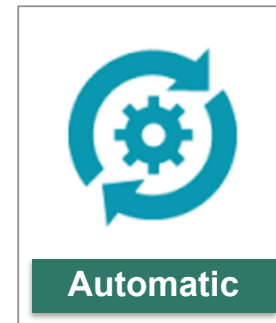
Efficiency



Timeliness

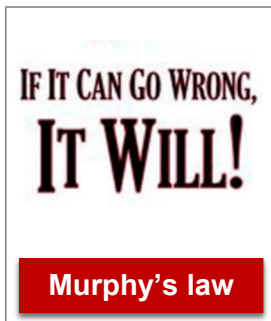


Trustworthiness

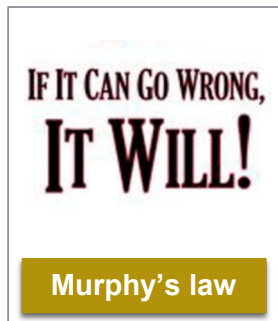


Automatic

Challenges



Murphy's law



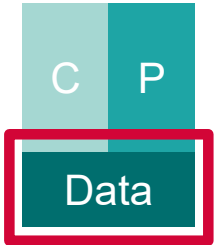
Murphy's law



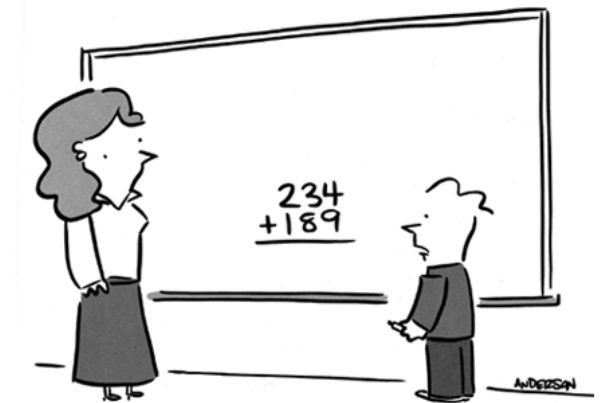
Overview of Chapter 1

-
- | | |
|-----|---|
| 1.1 | What are parallel and distributed systems? |
| 1.2 | ... from a Communication Perspective |
| 1.3 | ... from a Data Perspective |
| 1.4 | ... from a Processing Perspective |
| 1.5 | ... from a Privacy Perspective |
| 1.6 | Again: what are Parallel and Distributed Systems? |
-

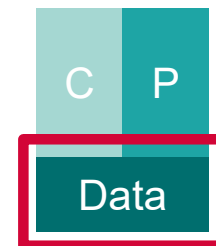
1.3 What are Distributed Systems? An Answer from a Data Perspective



- A Distributed System is a system in which (at least) the data layer is distributed (either logically –on the same computer system– or physically – across several systems connected via a network)
- Distributed Data Management is particularly relevant in the context of **Big Data**
 - “Data is all around”
 - The term Big Data is actually not new ...
 - ... but what people conceive as Big Data has changed significantly over the past decades



“Big Data” back in 1975



Example: Data management at the Dutch telecom PTT

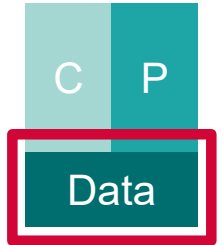
- Data size
 - 5 Mio. phone numbers
 - 4.5 Mio. subscribers
 - 0.275 Mio. work orders
 - 0.15 Mio. streets
 - 3'500 cities
- Data Access
 - 110'000 bills per day
 - 105'000 inquiries per day
- Updates
 - 110'000 payments per day
 - 10'000 subscriber updates per day
 - 2 DB copies (backup) per day
 - 1 reorganization per week (35'000 records)



Source: “Database Experience in the Netherlands PTT” (VLDB, 1975):



“Big Data” today

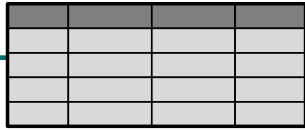


- Facebook
 - approx. 350 million of new photos per day
(numbers from 2020, source: <https://www.omnicoreagency.com/facebook-statistics/>)
 - More than 300 petabytes of data (source: <https://blog.wishpond.com/post/115675435109/40-up-to-date-facebook-facts-and-stats>)
 - approx. 100 million hours of video are watched every day
- Instagram (numbers from 2020, source <https://www.omnicoreagency.com/instagram-statistics/>)
 - 50+ billion photos in total
 - approx. 100+ million new photos per day
- WhatsApp
 - Over 100 billion messages per day
- Youtube (numbers from 2022, source: <https://www.omnicoreagency.com/youtube-statistics/>)
 - In total, 5+ billion videos have been shared (2020)
 - approx. 500 hrs. of new videos uploaded per minute
 - Total Daily Hours of Video Watched: 1 billion



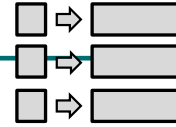
Evolution of Data Models

Relational



- Table representation
- Structured data in a strict schema
- Transactional workloads

Key-Value



- Array-like data
- High performance and scalability
- Allows storing arbitrary values

Document

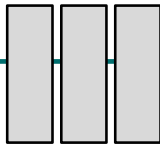
C P

Data

A diagram of a document database showing a stack of documents. The top document is highlighted with a red border and contains the word "Data". Above the stack are two small boxes labeled "C" and "P".

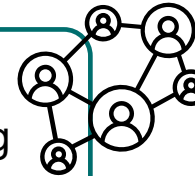
- Collection of documents
- Unstructured data / following no schema
- Nested structures

Analytical



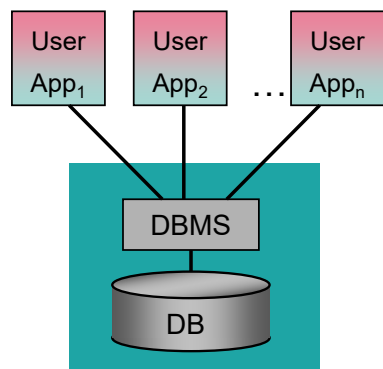
- Comes in various shapes, e.g., based on the relational model
- All items of a column are stored together

Labeled-Property Graph



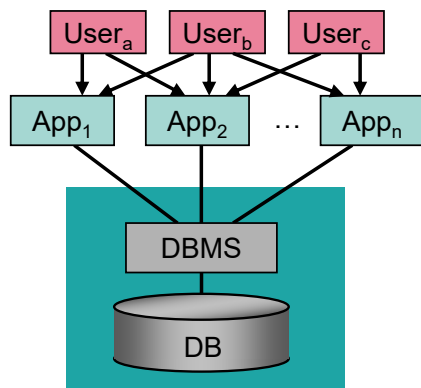
- Data is represented using nodes and edges
- Good for storing and querying relationships

(Simplified) Evolution of Database Architectures



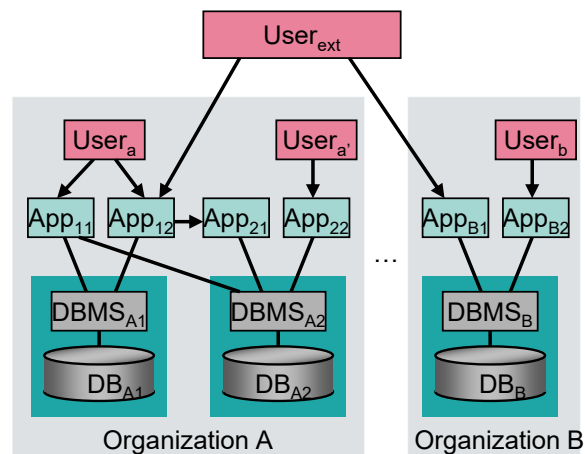
Two-tier
Architecture

Centralized
Data Management



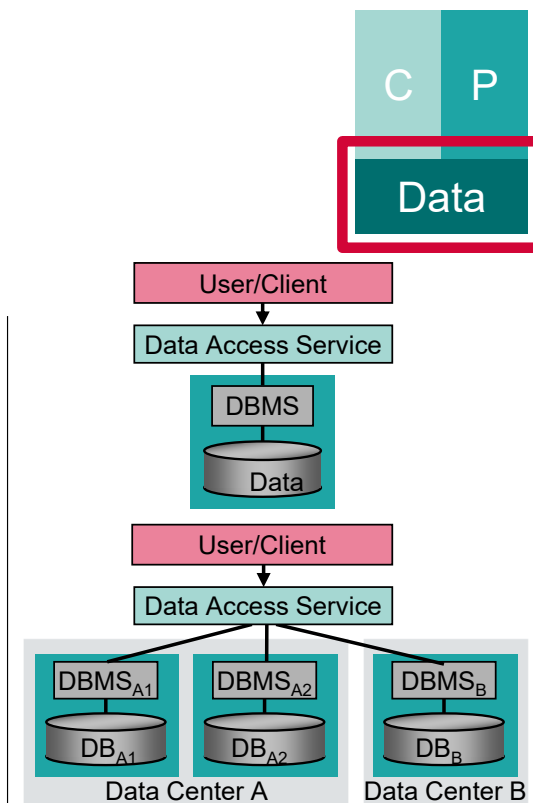
Three-/multi-tier
Architecture

Centralized
Data Management



Three-/Multi-Tier
Architecture

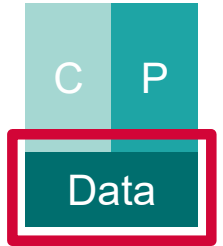
Distributed
Data Management



Appears like: Two-Tier,
Centralized Architecture

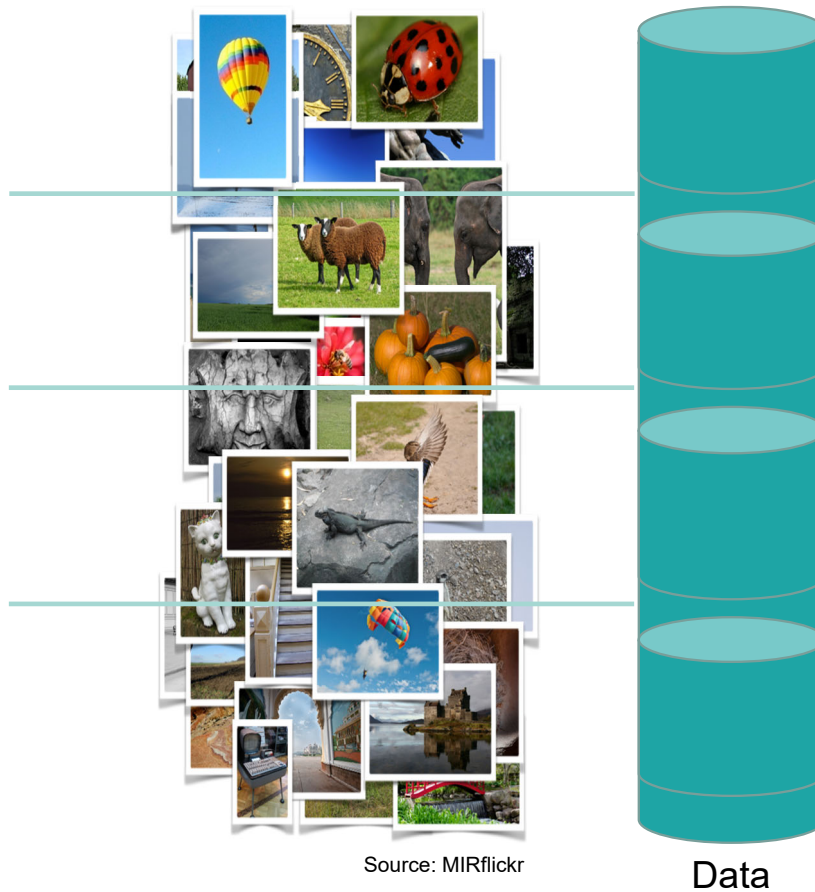
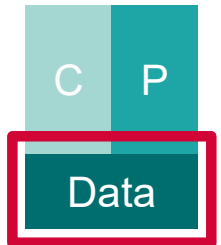
Actually is: Two-/Multi-Tier
Distributed
Data Management

Big Data: the Invariant



- Independent of the sheer size of “Big Data” (in their contemporary interpretation), there is one important invariant:
 - The volume of data is in most cases **too big to be managed on a single computer** (and the local storage attached to it)
 - Solution: split data in smaller chunks and distribute them across several computers (nodes in a distributed system)

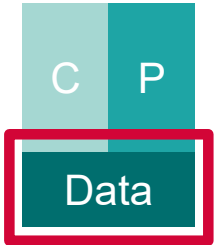
Explicit Distribution of Data (Data Sharding)



Distribution for

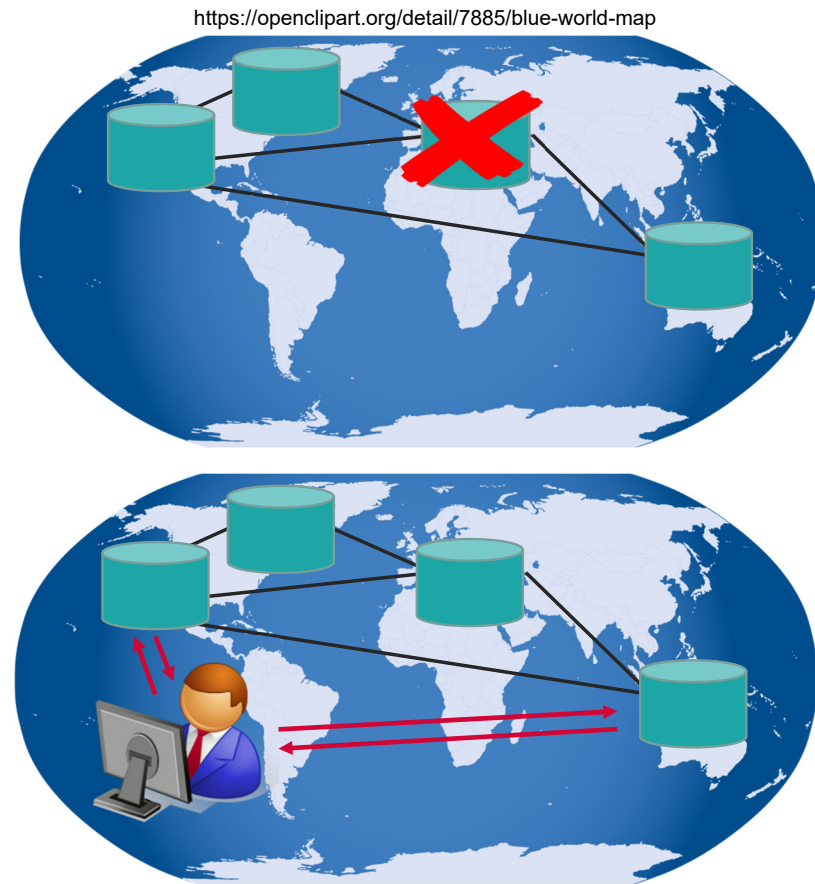
- Data management
- Data processing
(partitioning and/or replication,
for parallel access)

Data Distribution

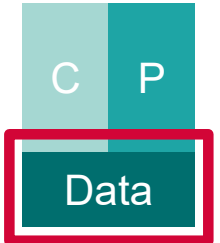


Further reasons for (global) data distribution

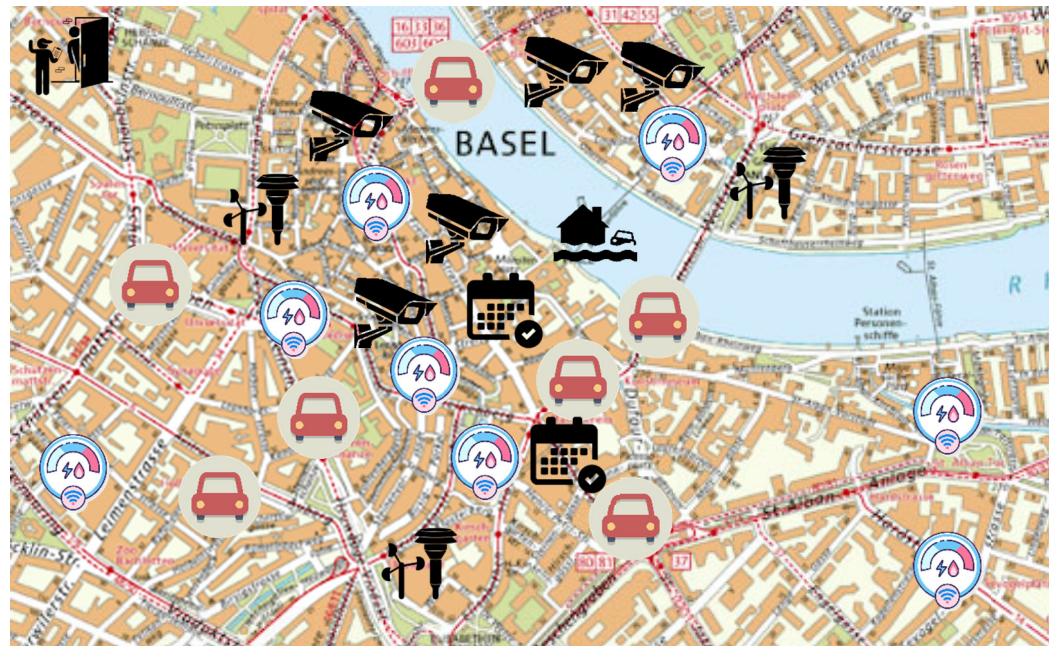
- **Fail-safe systems:** replicated data
- **Efficient access:** make data accessible close to their users



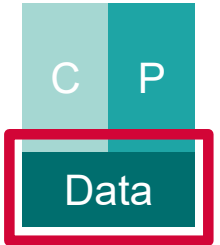
Integration of Data



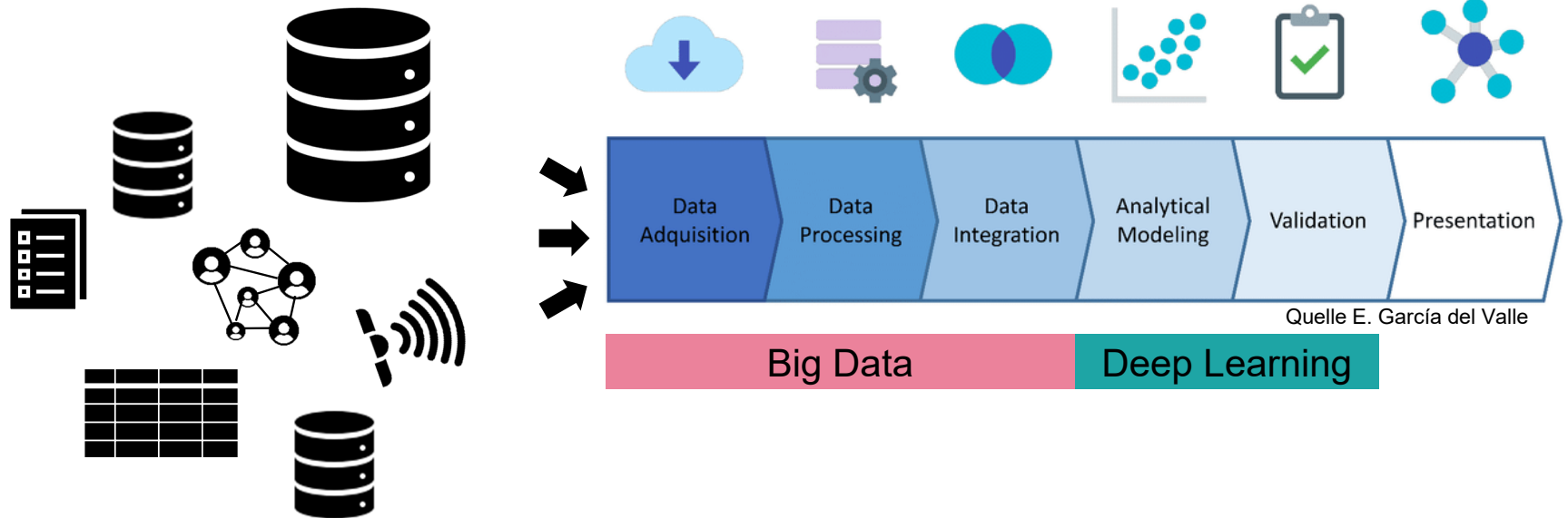
- However, in some cases the distribution of data is not a solution, but rather part of a challenge
- Data collections that have evolved naturally and independently over the years
 - Challenge: Integrate heterogeneous data to jointly analyze it



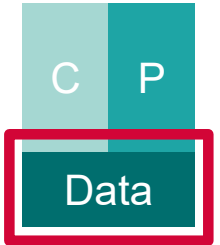
Big Data and Data Science



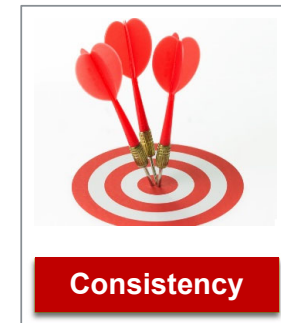
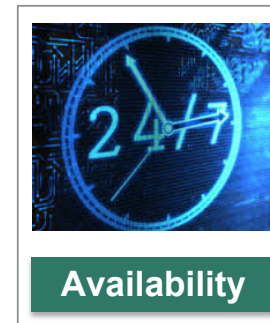
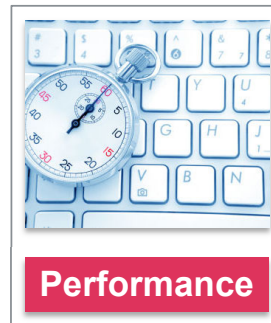
- Data Analytics is “only” the last step in the Data Science pipeline
- The first, essential part, needs to deal with Big Data, and in particular **distributed data**
 - Volume
 - Heterogeneity
 - Velocity



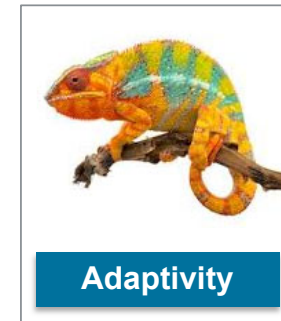
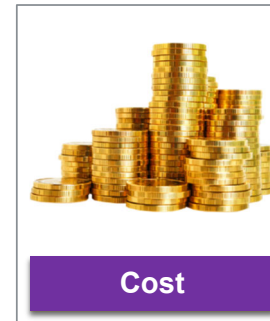
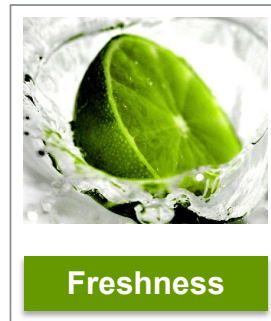
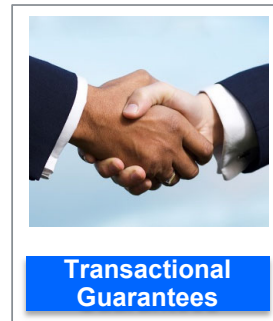
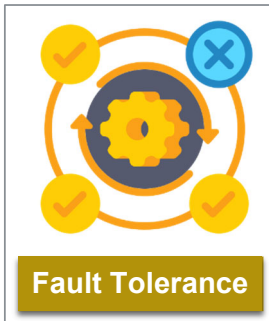
From a Data Perspective: Goals & Challenges of Distributed Systems



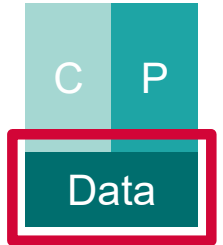
- Goals



- Challenges

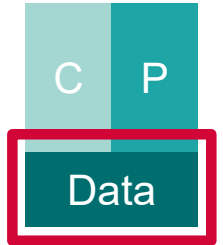


From a Data Perspective: Goals of Distributed Systems



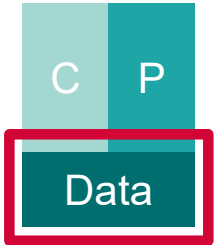
- Using a distributed system to manage data allows to cope with huge volumes and to **scale** the data management layer together with the application demands
- Distributing data also allows to **parallelize** the access to data and/or to balance the load in the system and thus to increase the overall **performance** of systems
- At the same time, managing data at several sites may also increase the **availability** of data in case of replication, but raises the challenge to guarantee the **consistency** of data (make sure that all copies of an object have the same value)

From a Data Perspective: Challenges of Distributed Systems

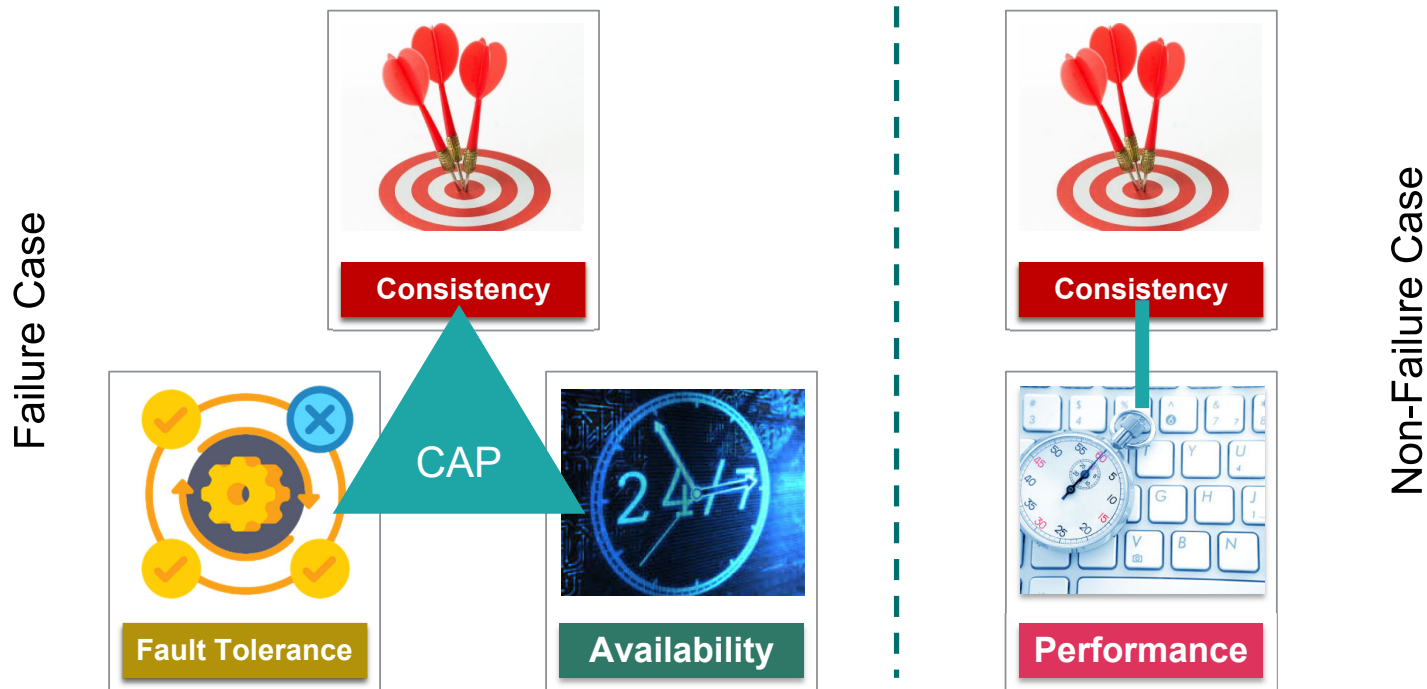


- Distributed Data Management comes with some major challenges
 - **Fault tolerance** – make sure that (partial) failures can be coped with correctly
 - **Transactional guarantees**: make sure that applications that span several sites respect the ACID properties of database transactions
 - **Data Freshness**: make sure that data, if available in several versions (freshness levels) is provided with at least the desired degree of freshness
 - **Cost**: provide all these features in a cost-optimized, economic way
 - **Adaptivity**: dynamically apply changes (e.g., to data distribution, coordination protocols, etc.) as the access characteristics to data changes
- Additional challenges: deal with
 - (structural/semantic) **heterogeneity**
 - ...

Dependencies



- Some of the goals / challenges are mutually exclusive
 - CAP Theorem: only two out of **C**onsistency, **A**vailability, **P**artition Tolerance
 - More precisely: PACELC: **CAP** (in failure case) **E**lse **L**atency or **C**onsistency



From a Data Perspective: Goals & Challenges of Distributed Systems

C P

Data

Chapter 4.3: Data Replication

Chapter 4.1: Self-Organization



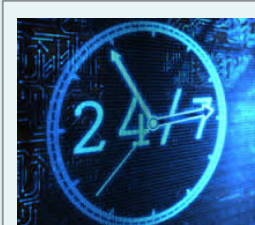
Scalability



Scalability



Performance



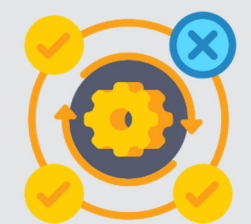
Availability



Consistency

(Chapter 3.5: Consensus)

Chapter 4.2: Consistency



Fault Tolerance



Transactional
Guarantees



Freshness



Cost



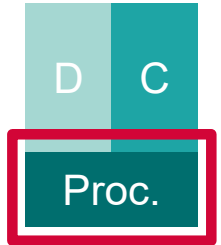
Adaptivity

Overview of Chapter 1

-
- | | |
|-----|---|
| 1.1 | What are parallel and distributed systems? |
| 1.2 | ... from a Communication Perspective |
| 1.3 | ... from a Data Perspective |
| 1.4 | ... from a Processing Perspective |
| 1.5 | ... from a Privacy Perspective |
| 1.6 | Again: what are Parallel and Distributed Systems? |
-

1.4 What are Distributed Systems?

An Answer from the **Processing of Computations** Perspective



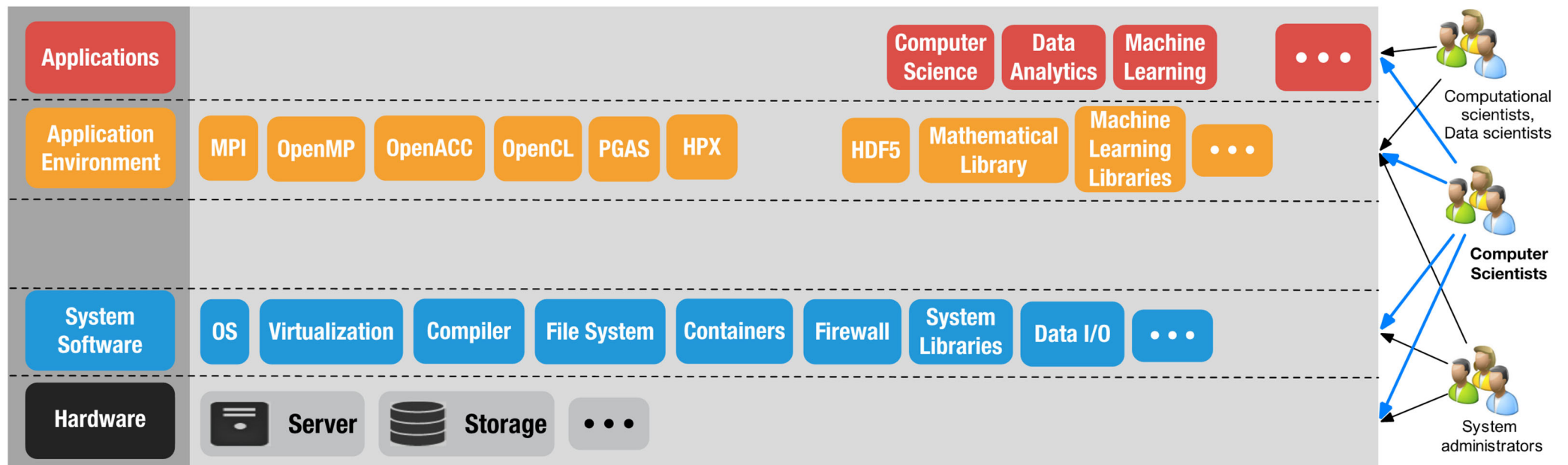
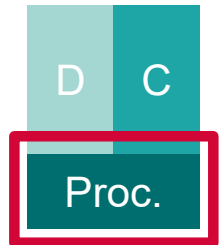
What are Systems that **Process Computations** (aka **Computing** Systems)?

- A **computer system** is the **ensemble** of the
 - Software (SW) and its architecture (SW-ARCH)
 - Hardware (HW) and its architecture (HW-ARCH)
- Characterized by the **interplay** between
 - Software components & architectures
 - Hardware components & architectures
 - System software & Tools
 - Applications
 - Services
- **Purpose**
 - Perform **computation** while seamlessly leveraging the available **data** and **communication** subsystems, under the threat of **privacy** violations.

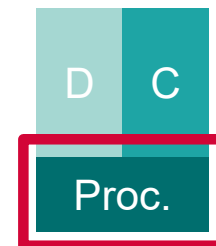
*“The purpose of computing is **insight**, not numbers.”*

R. Hamming
Numerical Methods for
Scientists and Engineers, 1973

Computing Ecosystem



What are **Parallel** & **Distributed Computing Systems**?

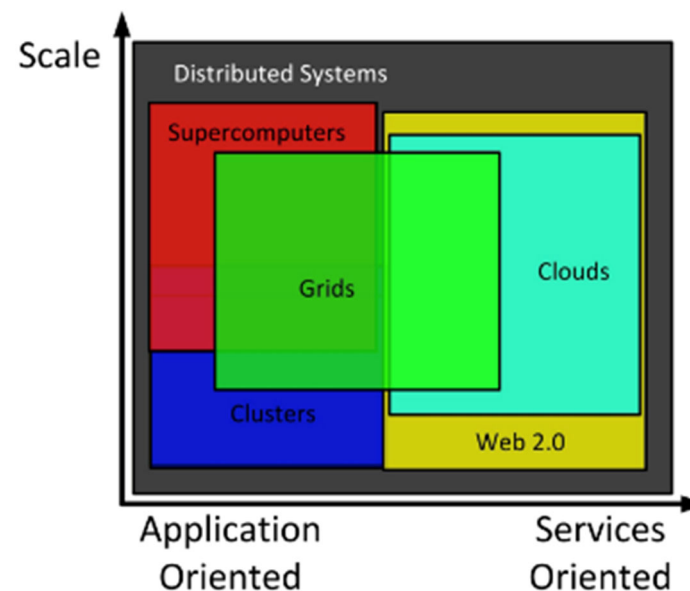


- **Parallel Computing Systems (PCS)**
 - Suitable for **tightly** coupled (synchronous) computations -> application oriented
 - Typically located within **geographical proximity**
 - With **shared** and/or distributed memory
 - Ex.: Clusters, Supercomputers, Grids

A **high performance computing (HPC) system** is a P&DCS system designed to *efficiently* solve a large problem, typically on more than a single computer (e.g., a Cluster, a Supercomputer, the Cloud).

A **supercomputer** is a **top class HPC** system concentrating the highest compute capacities on Earth, in ~10k servers connected by high-speed communication network.

- **Distributed Computing System**
 - Suitable for **loosely** coupled (asynchronous) computations -> services oriented
 - **Not** defined by geographical **proximity**
 - With shared and/or **distributed** memory
 - Ex.: Data Centers, Clouds



[Image: Ioan Raicu, Scalable Resource Management in Clouds and Grids, 2008]

High Performance **Parallel** & **Distributed** Computing

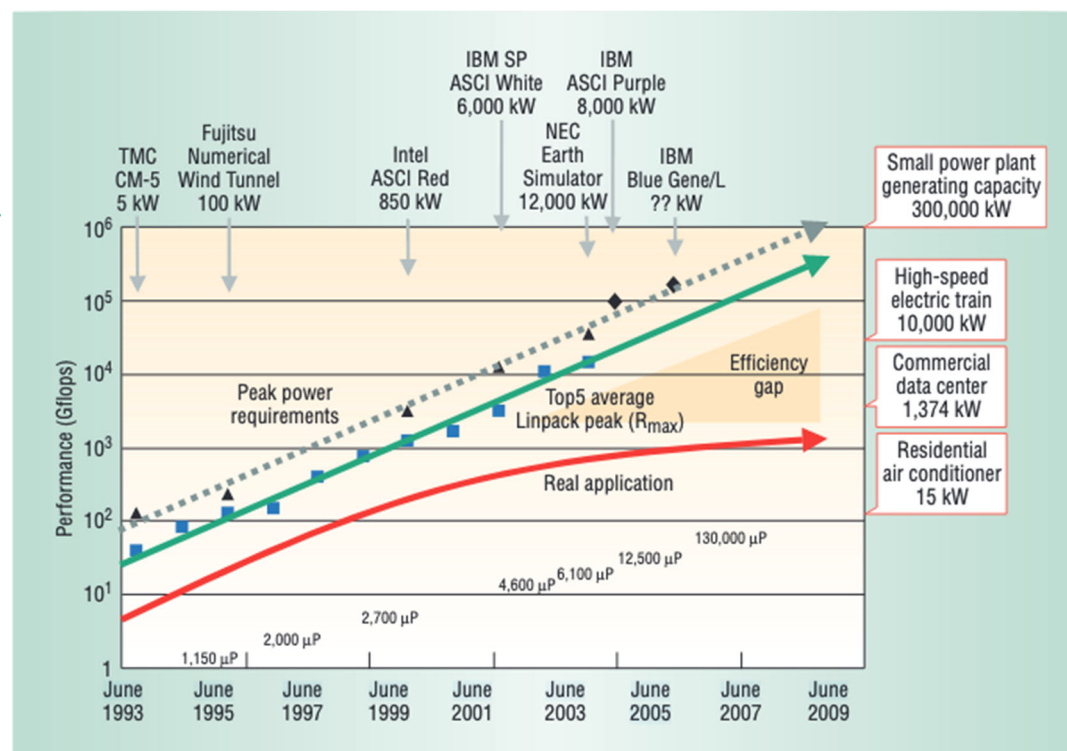
Processing Speed Magnitudes

D C

Proc.

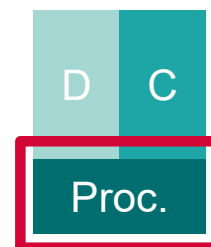
- Kilo = 10^3 , Mega = 10^6 , **Giga** = 10^9 , Tera = 10^{12} , **Peta** = 10^{15} , **Exa** = 10^{18} Flop/s
- **High Performance Computers** today
 - Desktop: **GigaFlop/s**
 - Service centers: TeraFlop/s or **PetaFlop/s**
 - Research: **ExaFlop/s** -> **Current HPC frontier**
 - **650 mW power requirements** $\approx 2\times$ small power plant
 - Each increase in order of magnitude is a big deal:

“[it is like going] from **walking** to explore one’s street, to **biking**, driving, and ultimately **flying** to explore the world”
(Dona Crawford)



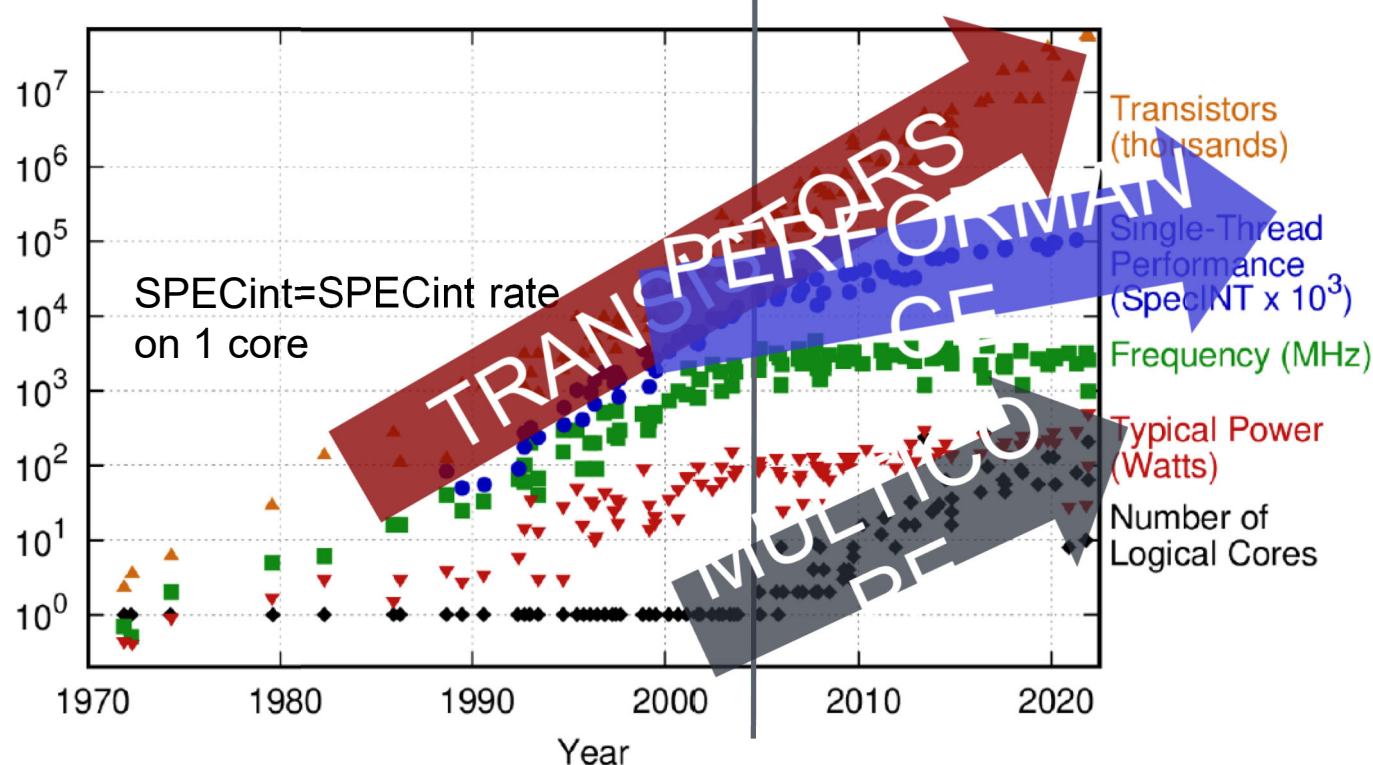
[Image: Cameron, K. W., Rong Ge, & Xizhou Feng. High-performance, power-aware distributed computing for scientific applications. Computer 38, 40–47 (2005)]

Evolution of HPC Systems over the last 50+ Years



SPECint rate on 2+ cores

50 Years of Microprocessor Trend Data



- **Moore's Law (1965):** The transistor count on a cost-effective integrated circuit doubles every 18–24 months.
- **Dennard's Scaling (1974):** Voltage and current should be proportional to the linear dimensions of a transistor.
- **End of Dennard's Scaling (2004):** Single-core performance plateauing after decades of exponential growth

-> **Multicore era (2005-today)**

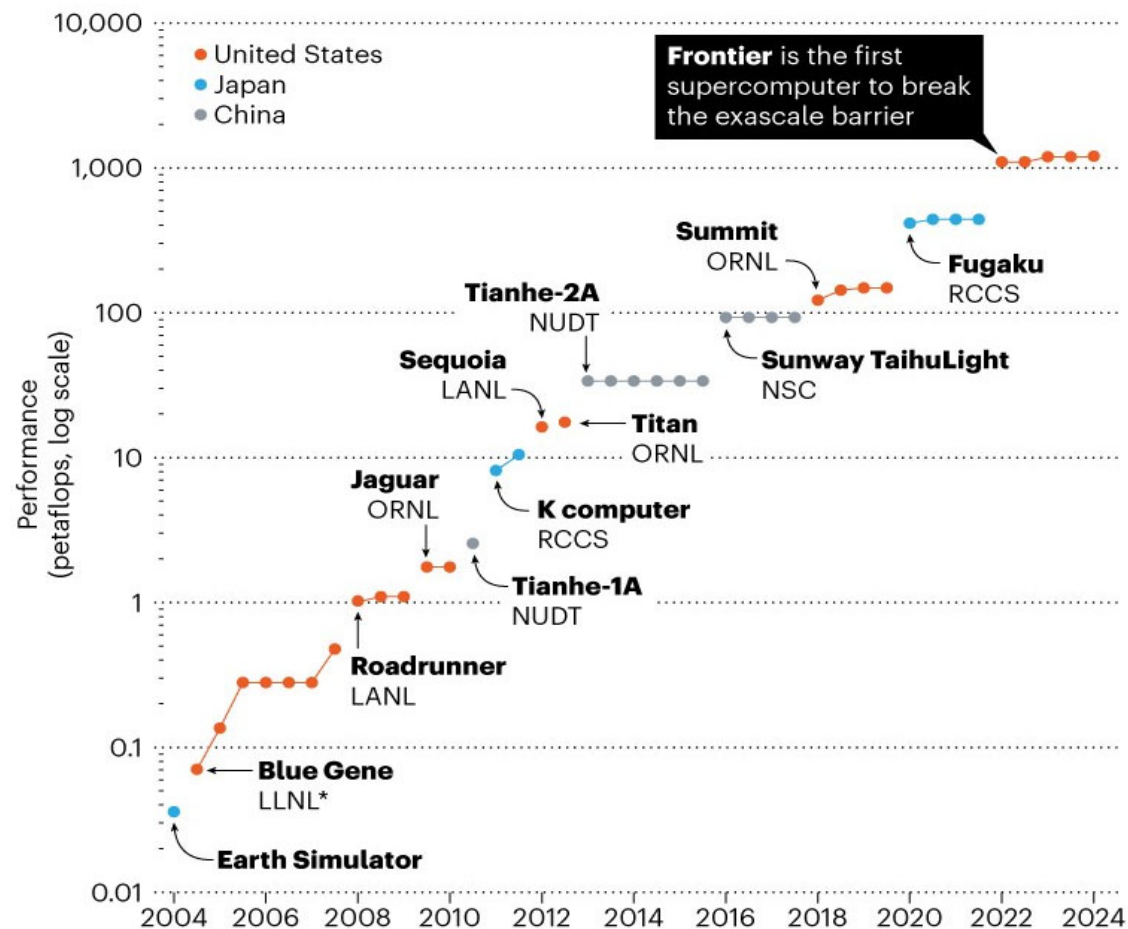
Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

[Adapted from Jack Dongarra, "HPC: Where We Are Today And A Look Into The Future", PPAM September 2022]

Evolution of HPC Systems Performance over the last 20+ Years

D C

Proc.



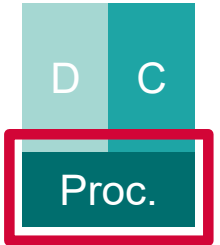
[S. Chen, "A day in the life of the world's fastest supercomputer", <https://www.nature.com/articles/d41586-024-02832-5>]

- **Speed Records:** Design advances over the past 20 years have produced huge increases in speed for the world's fastest supercomputers, rated using the HPL benchmark to generate the TOP500 list.
- **Exascale Supercomputer:** First reached by **Frontier** (at Oak Ridge National Laboratory) that performed **10¹⁸ Flop/s** for HPL in double precision
 - That's one quintillion operations per second by a single system!
 - El Capitan (LLNL) 1.75 EF/s



<https://top500.org/lists/top500/2024/06/>

Exascale Supercomputing Era

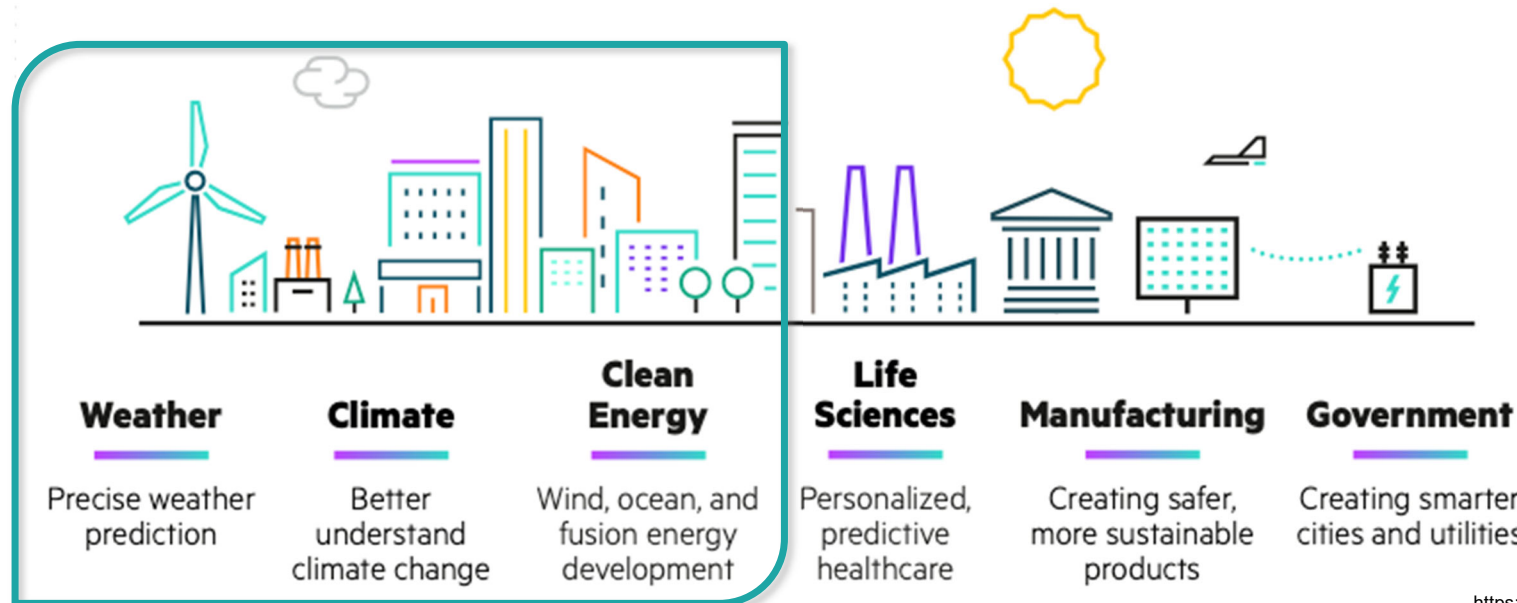


Start counting!

.....

If someone started counting one number per second, it would take almost 32 billion years to **count to one quintillion!**

What can we do with exascale supercomputers?



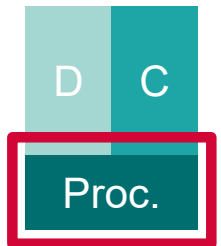
*“The purpose of computing is **insight**, not numbers.”*

R. Hamming
Numerical Methods for
Scientists and Engineers, 1973

https://www.hpe.com/psnow/doc/a50005129enw?jumpid=in_pdfviewer-psnow

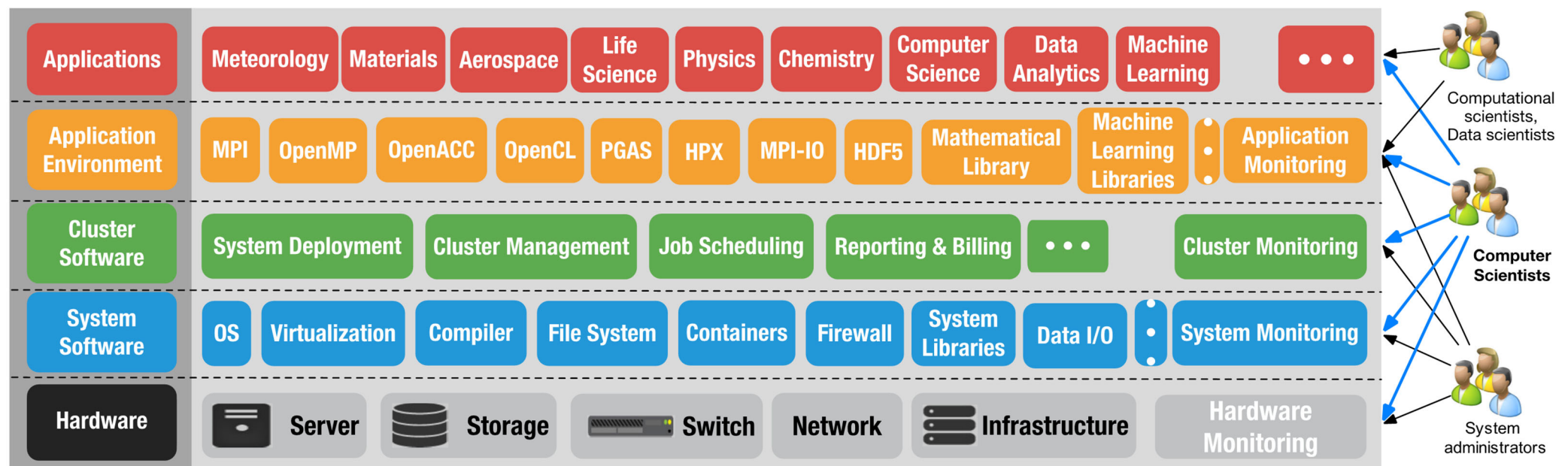
High Performance Computing | Supercomputing

Rich, Diverse, Complex Ecosystem



- “*Far more than* one sponsor, one site, one application, one software system, or one architecture. [...] it is a **community of interacting entities**.”

“Contemporary High Performance Computing: from Petascale toward Exascale”, 2019, p. xix



High Performance Computer | Supercomputer

A Parallel & Distributed System of Subsystems

- On-premises
- In the Cloud

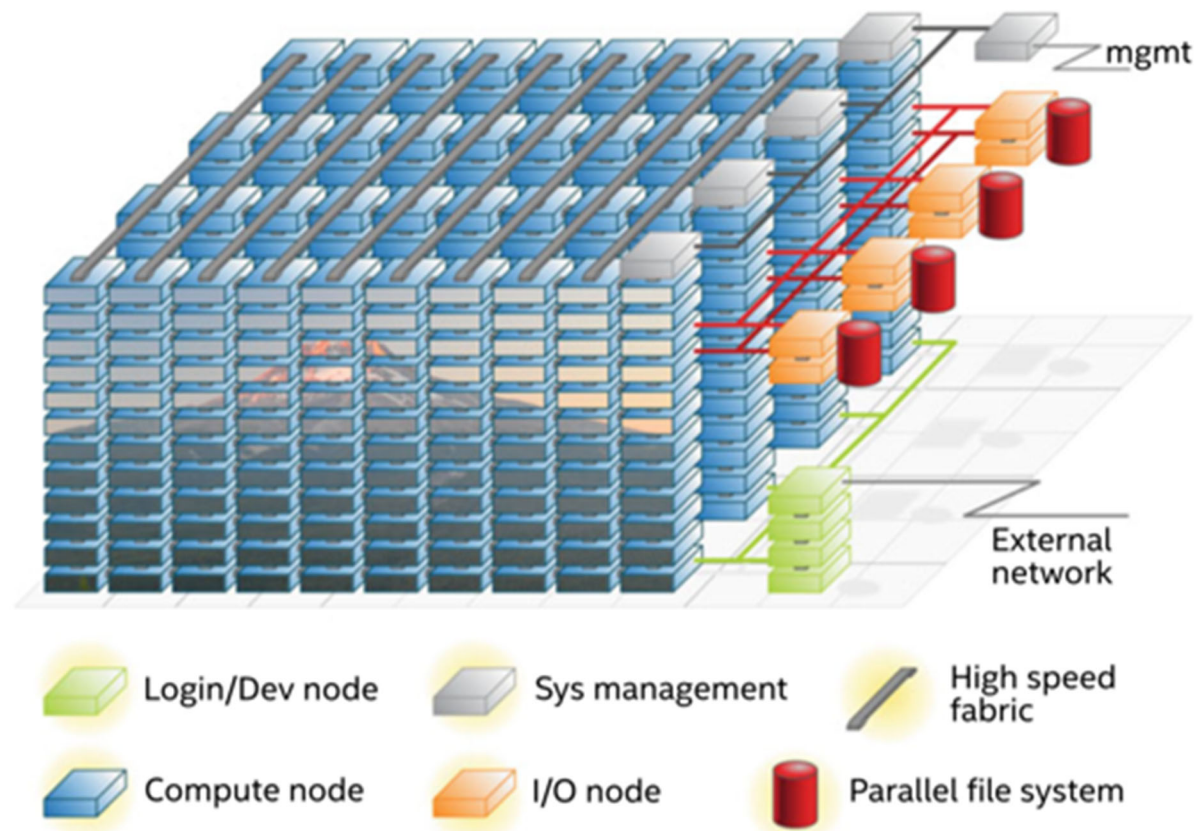


Fig. 1.2 Depiction of major, common subsystem partitioning in large HPC machines

[Source: B. Gerofi et al. (eds.), Operating Systems for Supercomputers and High Performance Computing, 2019]

D C

Proc.

D	C
Proc.	

```
00111101001010101
01001010100010101
01010ERROR011010
10100010100110001
01100010100100111
10100010100110001
```



The diagram illustrates a High-Performance Computing (HPC) zone. It features a large yellow rectangular area labeled "High-Performance Computing Zone" at the top. Inside this zone, there is a grid of 30 blue rectangular blocks, each representing a compute node. These nodes are arranged in 5 rows and 6 columns. Each node contains a small icon of a server rack and the number "10". Below the yellow zone, there is a light blue horizontal bar labeled "Compute Nodes".

The diagram illustrates a deeply heterogeneous processor architecture. The central processor block is divided into two main sections: ARM (big core) and ARM (lpg core). The ARM (big core) section includes NVM RAM, 3D-Stacked DRAM, and XBG blocks. The ARM (lpg core) section includes NVM RAM, 3D-Stacked DRAM, and XBG blocks. The processor is connected to external components: DR-Chip DRAM and DR-Chip NVM RAM.

The diagram illustrates the decomposition of a **Full Heterogeneous MetaProblem** into **Modules**, which are then further decomposed into **Task Parallelism** and **Data Parallelism**. The process involves defining multiple levels of modules and aggregating them to form a **Plan of Locality** and an **Aggregation** of memory hierarchy.

```

graph TD
    A[Full Heterogeneous MetaProblem] --> B[Module]
    A --> C[MetaProblem]
    A --> D[Module]
    A --> E[Component]
    A --> F[Module]
    B -.-> G[Module]
    C -.-> H[Task Parallelism]
    D -.-> I[Module]
    E -.-> J[Module]
    F -.-> K[Module]
    G -.-> L[Aggregate]
    H -.-> M[Data Parallelism]
    I -.-> M
    J -.-> M
    K -.-> M
    L -.-> M
    M --> N[Plan of Locality  
Elementary Computation]
    M --> O[Aggregation  
Splitting of Aggregates to  
Inner levels of  
Memory Hierarchy]
    
```

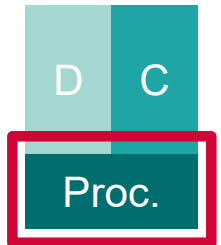
Figure 1: A hierarchical diagram showing the decomposition of a Full Heterogeneous MetaProblem into Modules, which are then further decomposed into Task Parallelism and Data Parallelism. The process involves defining multiple levels of modules and aggregating them to form a Plan of Locality and an Aggregation of memory hierarchy.

reproducibility
a principle of the scientific method

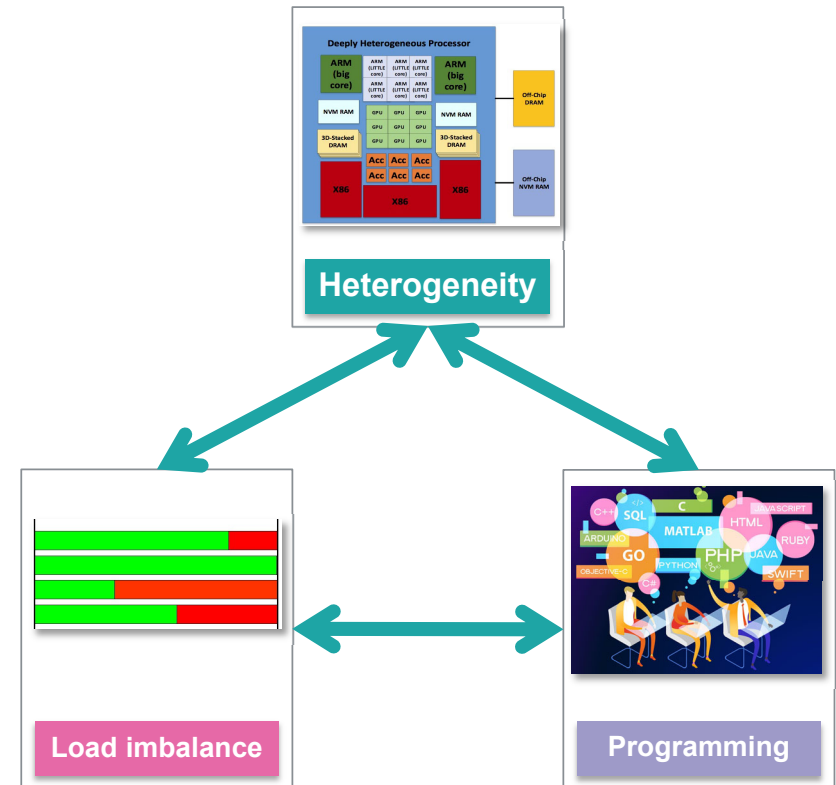
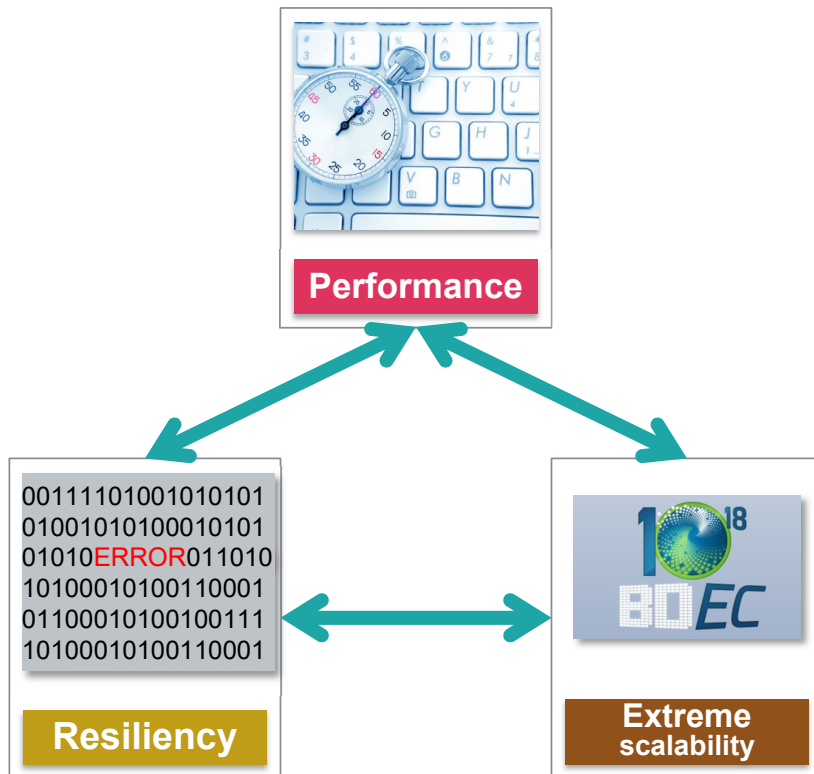
separates scientists from other researchers and normal people

1-40

Dependencies



- Some of these goals and challenges lead to tug-of-war situations **requiring trade-offs**
- **Performance** is the currency; **HPC** is a **trade-off space**



Dependencies

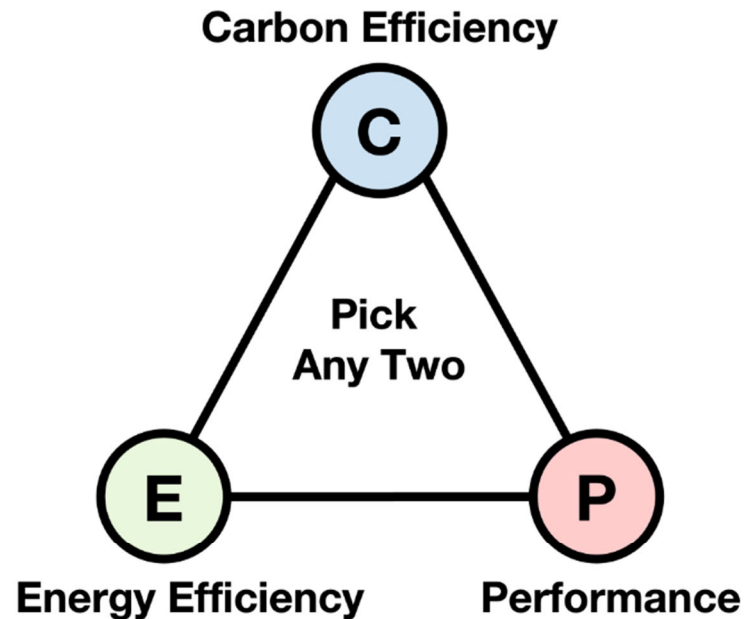
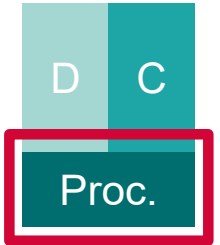


FIGURE 4. The CEP impossibility conjecture: “Pick any two of carbon efficiency, energy efficiency, and performance”.

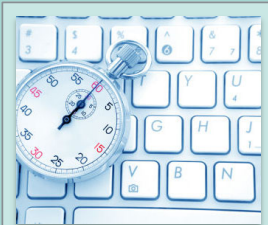
- **CEP Impossibility Conjecture**
- Under conditions where
- Energy is **not entirely** carbon-neutral and
- Carbon intensity **fluctuates** across **time** and **space**
- It is **unattainable** for systems to **concurrently optimize**
- **Carbon-efficiency**,
- **Energy-efficiency**, and
- **Performance**.

Addressed in The **Processing** Lecture Series

D C

Proc.

Chapter 5.1: Parallelization abstractions



Performance

```
00111101001010101
01001010100010101
01010ERROR011010
10100010100110001
01100010100100111
10100010100110001
```

Resiliency

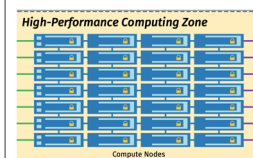
Chapter 5.2: Parallel architect- tures & models



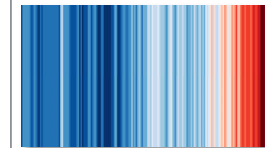
**Extreme
scalability**



Energy

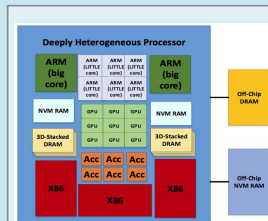


Security

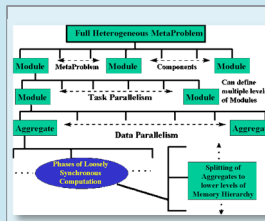


Sustainability

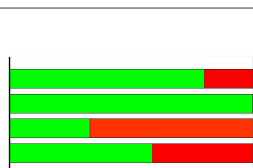
Chapter 5.3: Parallel programming



Heterogeneity



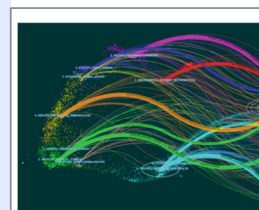
**Multilevel
parallelism**



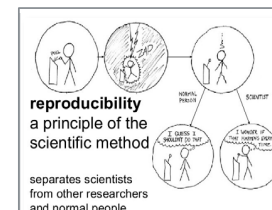
Load imbalance



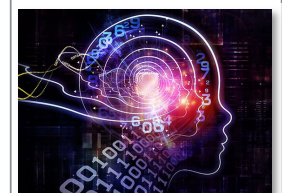
Programming



Visualization



Reproducibility



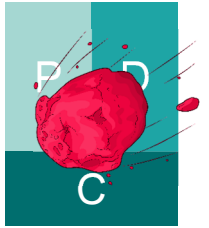
**Convergence
HPC+Data+AI**

Overview of Chapter 1

-
- | | |
|-----|---|
| 1.1 | What are parallel and distributed systems? |
| 1.2 | ... from a Communication Perspective |
| 1.3 | ... from a Processing Perspective |
| 1.4 | ... from a Data Perspective |
| 1.5 | ... from a Privacy Perspective |
| 1.6 | Again: what are Parallel and Distributed Systems? |
-

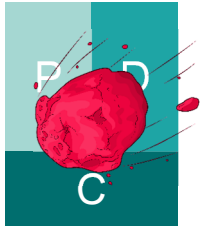
1.5 What are Distributed Systems?

An Answer from a **Privacy** Perspective

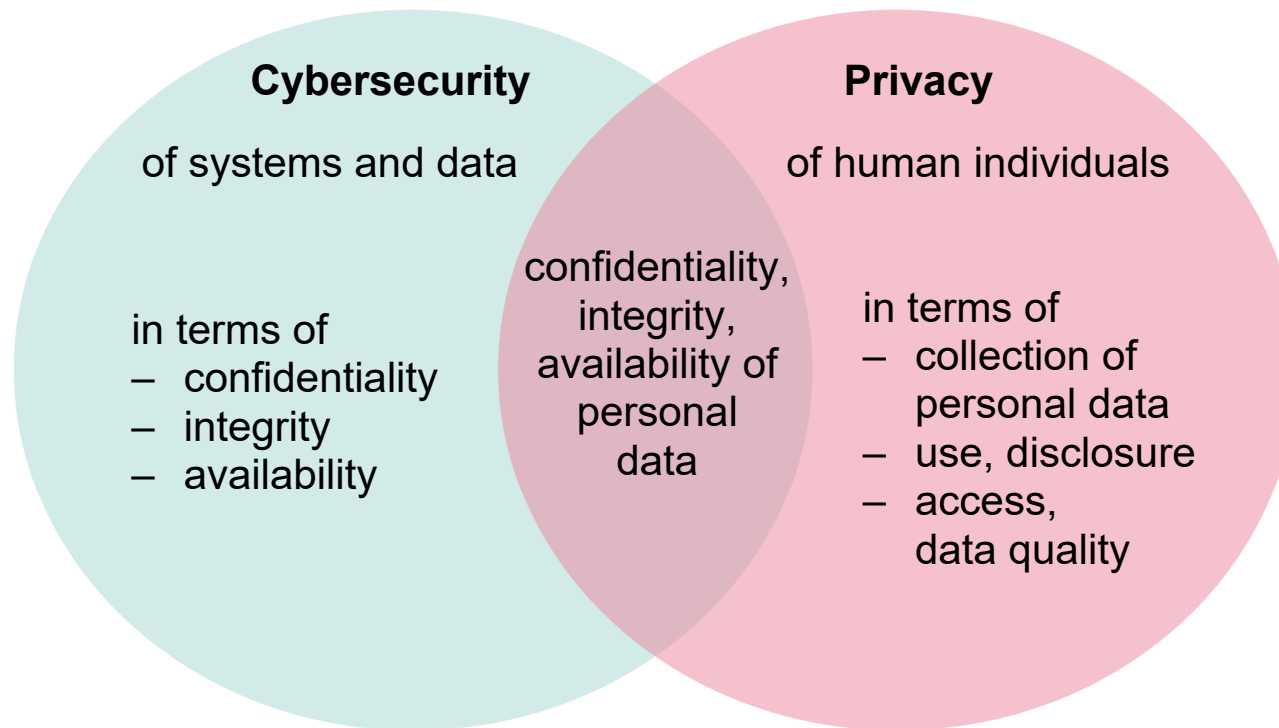


- Privacy is a **cross-cutting concern** in distributed systems
 - It affects all aspects of distributed system design: handling of data, design of communication, realization of processing
- Privacy is a **non-functional property**
 - To make a distributed system work, you absolutely have to figure out data, processing, and communication
 - But you can get away with disregarding privacy and still have a “working” system
- There are other cross-cutting non-functional properties (“asteroids”) besides privacy:
 - Security
 - Transparency
 - Usability
 - Environmental protection
 - ... And many more (see https://en.wikipedia.org/wiki/Non-functional_requirement)

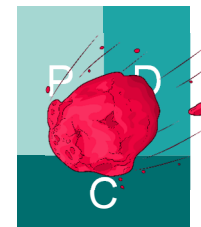
Privacy vs Security



- We focus on privacy because it is a fundamental human right
- There is some overlap with security (but only some!)



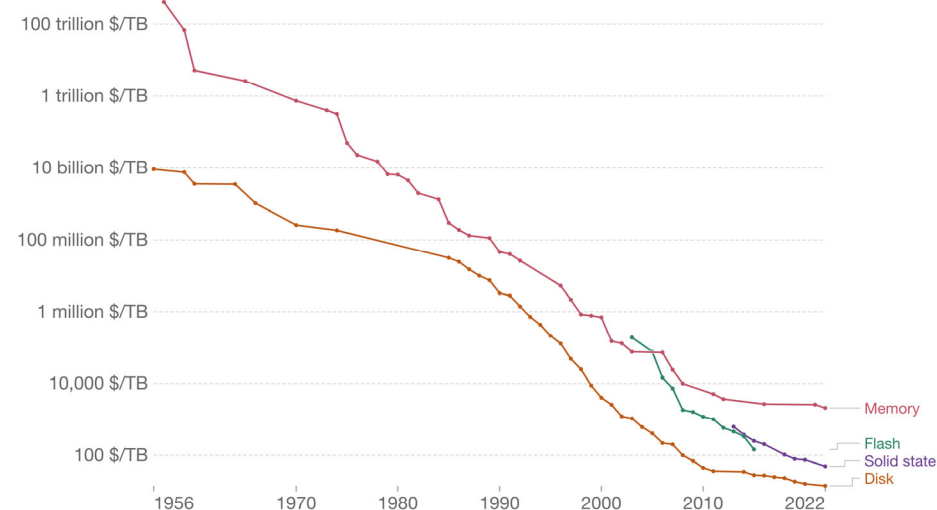
Privacy implications of cheap storage



- Decrease in storage cost leads to increased amount of data stored
- What kinds of data can now be stored without thinking twice, and essentially forever?
 - User behaviors, e.g., click traces, location history
 - Transaction histories
 - Video streams from surveillance cameras
 - History of search terms and resulting clicks
 - All video frames from your electric (self-driving) car

Historical cost of computer memory and storage

This data is expressed in US dollars per terabyte (TB). It is not adjusted for inflation.

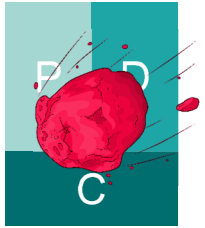


Source: John C. McCallum (2022)

Note: For each year, the time series shows the cheapest historical price recorded until that year.

OurWorldInData.org/technological-change • CC BY

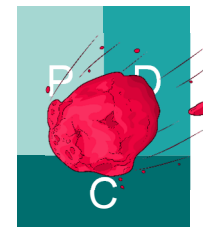
Privacy implications of big data



- Facebook
 - approx. 350 million of new photos per day
 - More than 300 petabytes of data
 - Approx. 100 million hours of video are watched every day
- Who owns these photos?
- Who can see them, and are users aware of this? (friends, friends of friends, the public)
- What does Facebook itself do with the photos?
 - Training of face recognition models?
 - Extract information from subjects and backgrounds?
- Does Facebook sell these models and inferred information?
- Photos of me can be uploaded without my knowledge



Privacy implications of increased compute power

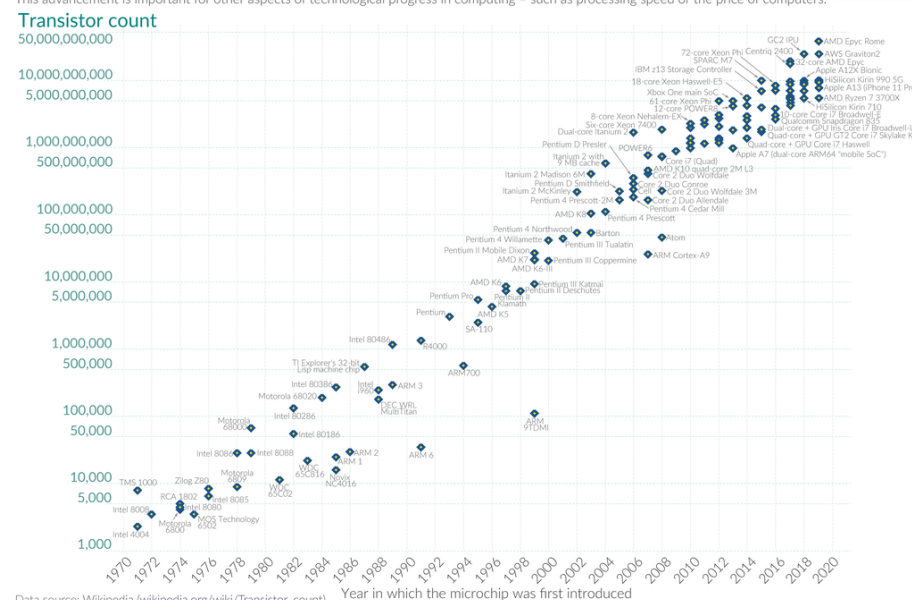


- Increase in compute capabilities means the stored data can be subjected to ever more analysis
- Inferences made by Facebook, based on user behavior, views/clicks (Kosinski 2013, PNAS):
 - sexual orientation
 - ethnicity
 - religious and political views
 - personality traits
 - intelligence
 - happiness
 - use of addictive substances
 - parental separation
 - age
 - gender

Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World
in Data

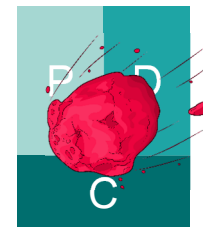


Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

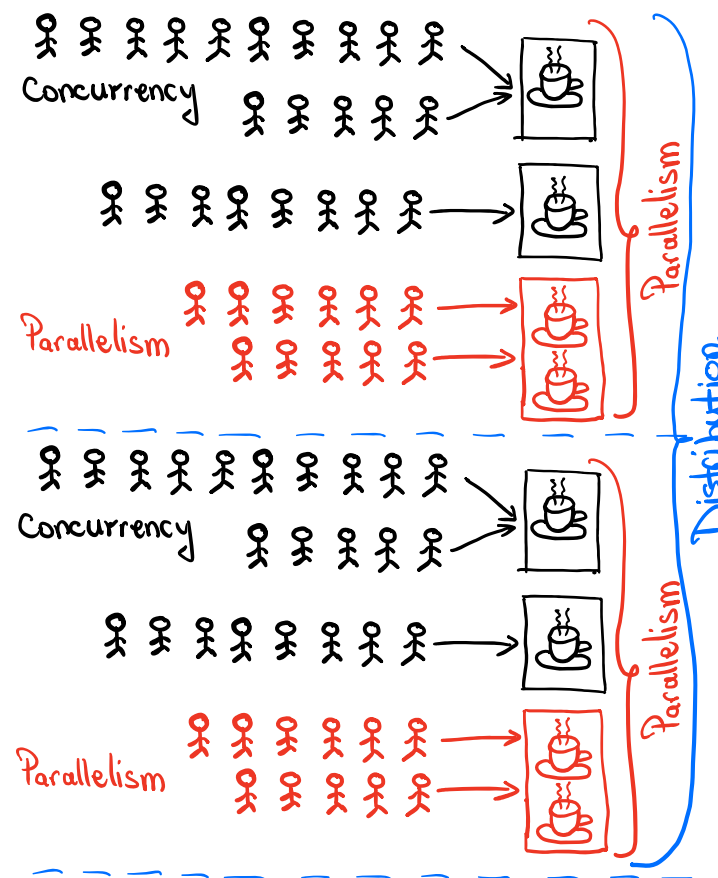
OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

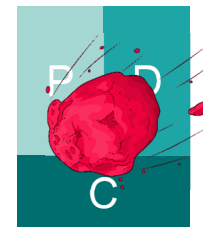
Privacy implications of (outsourced) computing



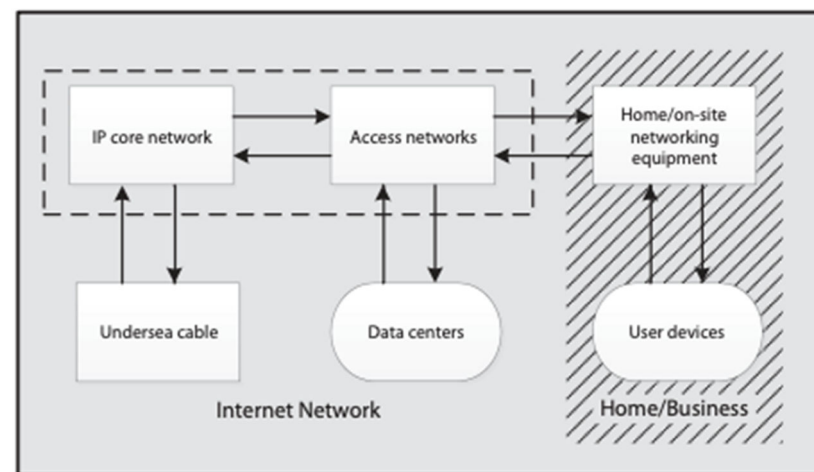
- What if the compute provider faithfully executes computations, but tries to learn additional information?
- What can a computing provider learn from observing compute jobs?
 - How often does someone enter the coffee queue?
 - At what times of day?
 - What drink does someone prefer?
 - At which geographic locations does someone use coffee queues?
 - What other patterns can we observe?
 - Are two users often served “together”?



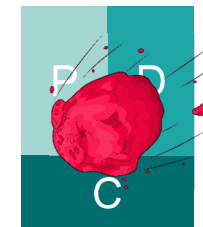
Privacy implications of networked communication



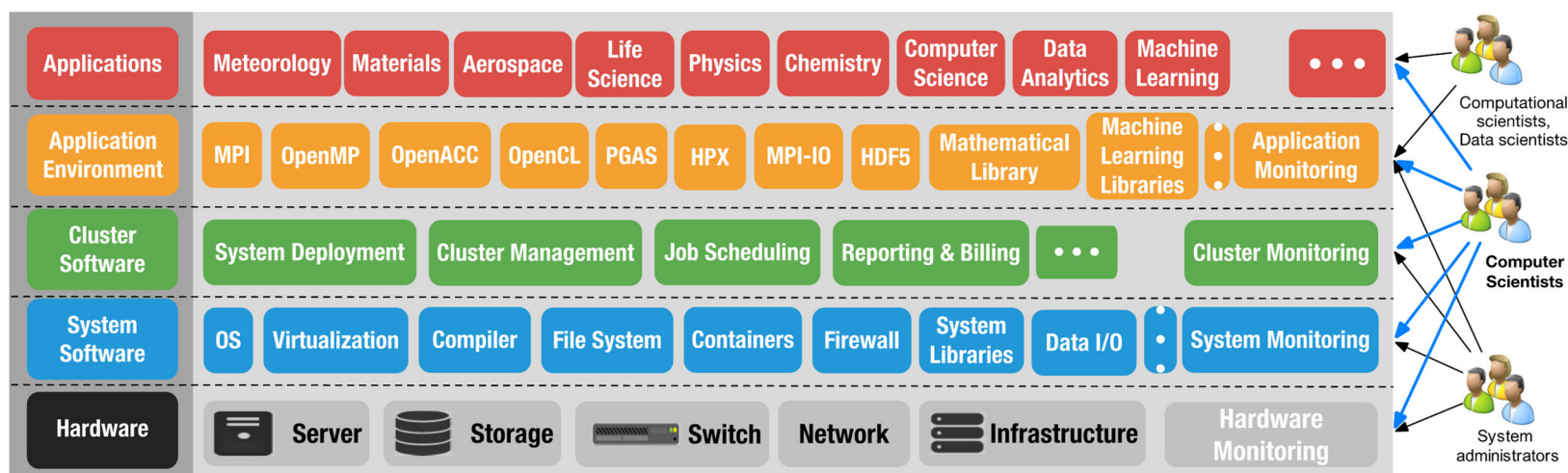
- Who operates this network equipment, and what information can they learn?
 - What if someone can observe traffic on an undersea cable?
 - On my wireless connection?
 - At my internet provider's access network?
- Securing traffic content is doable with modern cryptographic tools (such as TLS)
- Securing metadata is much harder
 - IP addresses are fairly unique identifiers, change infrequently
 - IP addresses can be geolocated, provider can link to user identity



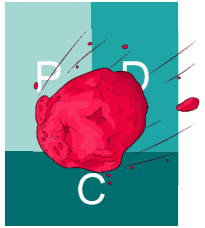
How can we address privacy threats?



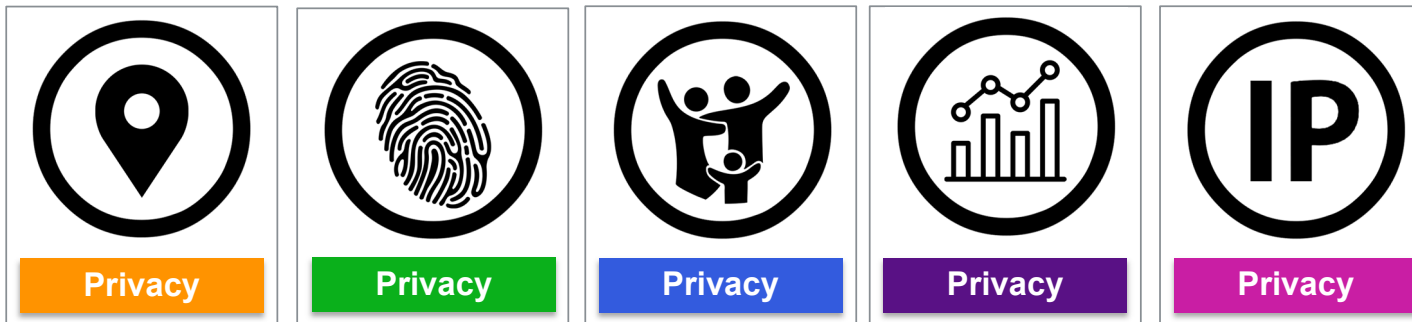
- Privacy mechanisms exist on all layers
- Abstractions are useful: lower-layer mechanisms provide **privacy-as-a-service** to higher layers
- But: problems and unintended side-effects often occur at the interfaces between layers
 - Do we understand the abstraction provided by the lower layer?
 - Is it the right abstraction for our application?



From a Privacy Perspective: Goals & Challenges of Distributed Systems



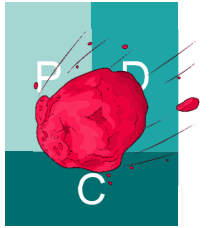
- Goals



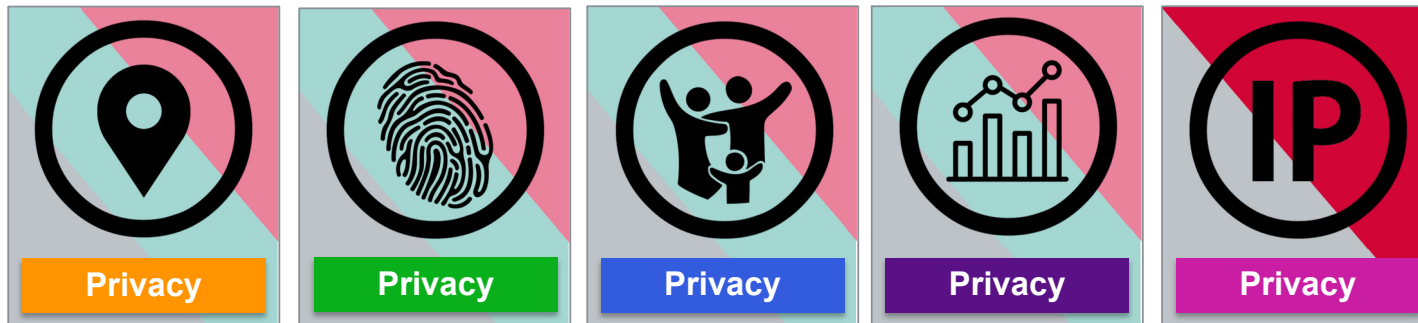
- Challenges



From a Privacy Perspective: Goals & Challenges of Distributed Systems



- Goals



- Challenges



Chapter 6.1: Introduction

Chapter 6.2: Differential privacy

Chapter 6.3: Homomorphic encryption

Chapter 6.4: Anonymous communication

Overview of Chapter 1

-
- | | |
|-----|---|
| 1.1 | What are parallel and distributed systems? |
| 1.2 | ... from a Communication Perspective |
| 1.3 | ... from a Processing Perspective |
| 1.4 | ... from a Data Perspective |
| 1.5 | ... from a Privacy Perspective |
| 1.6 | Again: what are Parallel and Distributed Systems? |

1.6 Again: What are the Challenges in Parallel and Distributed Systems?

Mastering Concurrency

- Align the goals and challenges ... from three different angles
- Work against impossibility theorems
- Carefully choose between conflicting goals, be aware of the consequences
- Have the right mindset!

Bachelor curriculum ~ (mostly) sequential programs

Master curriculum ~ (increasingly) concurrent, parallel, distributed programming

- Engineering choices and “Zeitgeist”:
Central control vs. decentralized operations